

SUNSHINE WALLET

API CONTRACT

Resident + Contributor + Council / Operator + Engine

Contract goal

Define the exact JSON shapes, endpoints, state transitions, calculation outputs and mock boundaries needed for the 25-hour, 3-person hackathon prototype. The API must make the engine explainable and keep frontend and backend developers independent.

Version 1.0 | 22 August 2026

MVP rule	Decision
Architecture	One Next.js web app; resident mobile-style view + Council desktop view.
Backend	Next.js route handlers/server actions; no microservices.
Persistence	Seeded TypeScript/JSON first. Optional Supabase only if it does not threaten the demo.
External systems	Smart meter, DNSP topology, retailer settlement and device control are mocked/simulated.
Real code	Window selection, eligibility, optimiser, simulation, M&V, contributor attribution, Equity Floor and settlement.
Truthfulness	Every simulated/estimated field exposes provenance.

1. Contract Scope and System Boundary

This API contract supports the complete demo loop: participant enrolment -> Sunshine Cell event -> optimiser -> simulated dispatch -> M&V -> settlement -> wallet credits. It intentionally does not model production market integrations.

Actor	What the API supports
Resident / equity participant	View wallet; see event explanation; register compatible flexible appliance; consent/pause/skip event.
Solar / flexibility contributor	Opt into participation; view qualifying contribution and contributor rewards; pause/skip.
Council / operator	View Sunshine Cells and forecasts; create/select event; optimise resources; run simulation; verify; settle; apply credits.
Engine	Deterministic, explainable calculations. No opaque ML dependency.
Partner adapter (mock)	Compatibility response, device dispatch acknowledgement, meter/export readings and settlement acknowledgement.

Critical modelling rule

Solar contributors and dispatchable flexible loads are not the same thing. Rooftop solar is measured as qualifying export during the event; the optimiser primarily selects flexible demand such as hot water, battery charging, EV charging and apartment common loads.

2. API Conventions

Convention	Contract
Base path	/api/v1
Media type	application/json
Time	ISO-8601 timestamps with timezone offset, e.g. 2026-08-23T12:00:00+10:00
Energy	kWh. Fields end with Kwh.
Power	kW. Fields end with Kw.
Money	Integer cents internally; optional formatted dollars in UI only.
Percentages	Decimals 0..1 in API, e.g. 0.82 = 82%.
IDs	Opaque strings; examples use evt_001, cell_dapto_01, hh_dinh.
Simulation	Every event and external response exposes simulation/provenance metadata.
Mutating settlement calls	Support Idempotency-Key to prevent duplicate credits.

3. Shared Enums and Provenance

Enum	Allowed values
ParticipantRole	EQUITY_PARTICIPANT, CONTRIBUTOR, BOTH
ParticipationStatus	ACTIVE, PAUSED, OPTED_OUT
ResourceType	STORAGE_HOT_WATER, COMMUNITY_BATTERY, EV_CHARGER, APARTMENT_COMMON_LOAD
CompatibilityStatus	PENDING, COMPATIBLE, INCOMPATIBLE
EventStatus	DRAFT, PROPOSED, OPTIMISED, DISPATCHED, VERIFIED, SETTLEMENT_BLOCKED, SETTLED, CREDITS_APPLIED, CANCELLED
Decision	SELECTED, PARTIALLY_SELECTED, REJECTED
RejectReason	NO_CONSENT, WRONG_CELL, UNAVAILABLE, INCOMPATIBLE, COMFORT_CONSTRAINT, LOW_CONFIDENCE, NOT_NEEDED
DataProvenance	REAL_PUBLIC, PARTNER_VERIFIED, ESTIMATED, SIMULATED, MOCKED
CreditType	EQUITY_DIVIDEND, CONTRIBUTOR_REWARD

```
{
  "provenance": {
    "networkTopology": "SIMULATED",
    "meterReadings": "SIMULATED",
    "compatibilityCheck": "MOCKED",
    "optimiserResult": "CALCULATED",
    "verificationResult": "CALCULATED",
    "settlementRate": "SIMULATED"
  }
}
```

Why provenance is in the API

The frontend should never have to guess which numbers are real, estimated or simulated. Judges should be able to see this truthfulness directly in the product.

4. Core Domain Objects

Object	Key fields
Participant	id, displayName, householdId, role, participationStatus, equityEligible
SunshineCell	id, displayName, suburb, topologyRef, equityScore, confidence, provenance
Resource	id, householdId, cellId, type, maxPowerKw, maxShiftEnergyKwh, availability, compatibility, consent, comfortSafe
SolarContributor	participantId, cellId, exportMeterRef, participationStatus
FlexEvent	id, cellId, startTime, endTime, targetFlexEnergyKwh, forecastConfidence, status
DispatchDecision	resourceId, score, scheduledEnergyKwh, decision, reason
Verification	baselineEnergyKwh, observedEnergyKwh, verifiedFlexEnergyKwh, confidence, settlementAllowed
Settlement	verifiedProgramValueCents, equityPoolCents, contributorPoolCents, reserveCents, policyVersion
Credit	participantId, eventId, type, amountCents, reason, status

5. Canonical TypeScript Interfaces

```

type DataProvenance = 'REAL_PUBLIC' | 'PARTNER_VERIFIED' | 'ESTIMATED' | 'SIMULATED' | 'MOCKED';

type ParticipantRole = 'EQUITY_PARTICIPANT' | 'CONTRIBUTOR' | 'BOTH';
type ParticipationStatus = 'ACTIVE' | 'PAUSED' | 'OPTED_OUT';

type ResourceType =
  | 'STORAGE_HOT_WATER'
  | 'COMMUNITY_BATTERY'
  | 'EV_CHARGER'
  | 'APARTMENT_COMMON_LOAD';

interface Resource {
  id: string;
  householdId?: string;
  sunshineCellId: string;
  type: ResourceType;
  maxPowerKw: number;
  maxShiftEnergyKwh: number;
  available: boolean;
  compatibilityStatus: 'PENDING' | 'COMPATIBLE' | 'INCOMPATIBLE';
  consentActive: boolean;
  comfortSafe: boolean;
  responseFactor: number; // deterministic simulation factor, 0..1
  scores: {
    networkEffectiveness: number; // 0..1
    equityNeed: number;           // 0..1
    controllability: number;       // 0..1
    rotationFairness: number;     // 0..1
  };
}

interface FlexEvent {
  id: string;
  sunshineCellId: string;
  startTime: string;
  endTime: string;
  targetFlexEnergyKwh: number;
  forecastConfidence: number;
  status: EventStatus;
  simulationMode: boolean;
}

```

6. Event State Machine

DRAFT

- > PROPOSED
- > OPTIMISED
- > DISPATCHED
- > VERIFIED
 - > SETTLED -> CREDITS_APPLIED
 - > SETTLEMENT_BLOCKED (e.g. M&V confidence < threshold)

Any pre-credit state may become CANCELLED when rules fail.

Transition	Required condition
PROPOSED -> OPTIMISED	Forecast gate passes; sufficient eligible resources.
OPTIMISED -> DISPATCHED	Operator runs simulated event.
DISPATCHED -> VERIFIED	Baseline and observed event data exist.
VERIFIED -> SETTLED	M&V confidence >= threshold AND Equity Floor policy valid.
VERIFIED -> SETTLEMENT_BLOCKED	M&V confidence below threshold or settlement policy invalid.
SETTLED -> CREDITS_APPLIED	Credit ledger write succeeds exactly once.

7. Endpoint Summary

Method	Path	Purpose
GET	/participants/:id	Participant profile and current participation status.
PATCH	/participants/:id/participation	Activate/pause/opt out.
POST	/resources/compatibility-check	Mock authorised-party compatibility check.
POST	/resources	Register an eligible flexible resource.
GET	/participants/:id/wallet	Resident/contributor wallet summary.
GET	/participants/:id/contributions	Contributor event history and rewards.
GET	/cells	Council opportunity list.
GET	/cells/:id	Cell detail and current forecast.
POST	/cells/:id/event-candidates	Calculate candidate windows and proposed event.
GET	/events/:id	Canonical event state.
POST	/events/:id/optimise	Filter, score and select flexible resources.
POST	/events/:id/simulate	Run deterministic simulated dispatch.
POST	/events/:id/verify	Calculate baseline, verified flexibility and M&V confidence gate.
POST	/events/:id/settle	Calculate value, Equity Floor, equity/contributor allocations.
POST	/events/:id/apply-credits	Write wallet credits exactly once.
POST	/events/:id/reset	Demo-only reset to deterministic starting state.

8. Resident and Contributor API

GET /api/v1/participants/:participantId [REAL]

Purpose: Return participant identity, role, equity eligibility and current participation state.

Used by: Resident / contributor UI

```
{
  "participant": {
    "id": "p_dinh",
    "displayName": "Dinh",
    "householdId": "hh_dinh",
    "role": "BOTH",
    "equityEligible": true,
    "participationStatus": "ACTIVE",
    "sunshineCellId": "cell_dapto_01"
  }
}
```

PATCH /api/v1/participants/:participantId/participation [REAL]

Purpose: Activate, pause or opt out of future qualifying events.

Used by: Resident / contributor UI

```
{
  "participationStatus": "PAUSED",
  "reason": "USER_REQUEST"
}
```

Response: participant object with updated status and effectiveAt timestamp.

POST /api/v1/resources/compatibility-check [MOCKED]

Purpose: Represent the authorised retailer/metering-provider compatibility check. This endpoint is MOCKED in the hackathon.

Used by: Resident onboarding

```
{
  "householdId": "hh_dinh",
  "deviceType": "STORAGE_HOT_WATER"
}
```

```
{
  "checkId": "compat_001",
  "provider": "SIMULATED_AUTHORIZED_PARTNER",
  "compatible": true,
  "maxPowerKw": 3.6,
  "maxShiftEnergyKwh": 5.0,
  "controlAvailable": true,
  "provenance": "MOCKED"
}
```

POST /api/v1/resources [REAL]

Purpose: Register a resource only after a successful compatibility check and explicit consent.

Used by: Resident onboarding

```
{
  "householdId": "hh_dinh",
  "sunshineCellId": "cell_dapto_01",
  "type": "STORAGE_HOT_WATER",
  "compatibilityCheckId": "compat_001",
  "consentActive": true
}
```

GET /api/v1/participants/:participantId/wallet [REAL]

Purpose: Return role-aware wallet totals and latest credit explanations.

Used by: Resident / contributor wallet

```
{
  "participant": {
    "id": "p_dinh",
    "displayName": "Dinh",
    "role": "BOTH"
  },
  "month": {
    "equityDividendCents": 1180,
    "contributorRewardCents": 235,
    "totalBenefitCents": 1415
  },
  "latestCredits": [
    {
      "eventId": "evt_001",
      "type": "EQUITY_DIVIDEND",
      "amountCents": 415,
      "reason": "Share of verified DAPTO-01 event value",
      "status": "APPLIED"
    }
  ],
  "participation": {
    "status": "ACTIVE",
    "canPause": true
  }
}
```

GET /api/v1/participants/:participantId/contributions [REAL]

Purpose: Return contributor export/accounting attribution and contributor reward history.

Used by: Contributor wallet

```
{
  "participantId": "p_maria",
  "events": [
    {
      "eventId": "evt_001",
      "measuredExportKwh": 6.5,
      "qualifyingExportKwh": 6.5,
      "contributorShare": 0.5,
      "attributedAccommodationKwh": 5.0,
      "rewardCents": 271,
      "provenance": "SIMULATED"
    }
  ]
}
```

Do not expose sensitive eligibility detail

The MVP should store only a simple eligibility result/token (for example equityEligible: true). Do not model welfare records or detailed income data.

9. Sunshine Cell and Event Discovery API

GET /api/v1/cells [REAL]

Purpose: Return Council opportunity cards.

Used by: Council dashboard

```
{
  "cells": [
    {
      "id": "cell_dapto_01",
      "displayName": "DAPTO-01",
      "suburb": "Dapto",

```

```

    "equityScore": 0.84,
    "solarOpportunityScore": 0.94,
    "networkNeedScore": 0.88,
    "confidence": 0.82,
    "priorityScore": 0.89,
    "topologyProvenance": "SIMULATED"
  }
]
}

```

GET /api/v1/cells/:cellId [REAL]

Purpose: Return detailed 24-hour forecast, resource availability summary and provenance.

Used by: Council cell dashboard

```

{
  "cell": {
    "id": "cell_dapto_01",
    "displayName": "DAPTO-01"
  },
  "forecast": {
    "intervalMinutes": 30,
    "provenance": "SIMULATED",
    "intervals": [
      {
        "start": "2026-08-23T12:00:00+10:00",
        "forecastExportEnergyKwh": 18.0,
        "comfortableExportEnergyKwh": 12.0,
        "availableFlexEnergyKwh": 16.0,
        "confidence": 0.84
      }
    ]
  },
  "resources": {
    "eligibleCount": 4,
    "availableFlexEnergyKwh": 31.0
  },
  "contributors": {
    "activeCount": 3,
    "forecastQualifyingExportKwh": 13.0
  }
}

```


POST /api/v1/cells/:cellId/event-candidates [REAL]

Purpose: Evaluate candidate windows and return the highest-scoring valid event or NO_EVENT.

Used by: Council dashboard + window selector

```
{
  "candidateWindows": [
    {
      "startTime": "2026-08-23T10:00:00+10:00",
      "endTime": "2026-08-23T12:00:00+10:00"
    },
    {
      "startTime": "2026-08-23T11:00:00+10:00",
      "endTime": "2026-08-23T13:00:00+10:00"
    },
    {
      "startTime": "2026-08-23T12:00:00+10:00",
      "endTime": "2026-08-23T14:00:00+10:00"
    },
    {
      "startTime": "2026-08-23T13:00:00+10:00",
      "endTime": "2026-08-23T15:00:00+10:00"
    }
  ],
  "minimumForecastConfidence": 0.7
}
```

```
{
  "decision": "EVENT_PROPOSED",
  "event": {
    "id": "evt_001",
    "sunshineCellId": "cell_dapto_01",
    "startTime": "2026-08-23T12:00:00+10:00",
    "endTime": "2026-08-23T14:00:00+10:00",
    "forecastExportEnergyKwh": 70.0,
    "comfortableExportEnergyKwh": 46.0,
    "targetFlexEnergyKwh": 24.0,
    "availableFlexEnergyKwh": 31.0,
    "forecastConfidence": 0.82,
    "windowScore": 0.91,
    "status": "PROPOSED"
  },
  "calculation": {
    "targetFlexEnergyKwh": "max(0, forecastExportEnergyKwh - comfortableExportEnergyKwh)",
    "windowWeights": {
      "solarOpportunity": 0.4,
      "networkNeed": 0.4,
      "availableFlexibility": 0.2
    }
  }
}
```

Target 24 kWh is calculated

The API should not treat 24 kWh as a magic constant. In the demo it is derived from simulated export opportunity minus simulated comfortable export level: $70 - 46 = 24$ kWh.

10. Optimiser API

POST /api/v1/events/:eventId/optimise [REAL]

Purpose: Apply hard eligibility gates, calculate explainable scores, rank resources and dispatch only enough energy to meet the event target.

Used by: Council optimiser screen

```
{
  "weights": {
    "networkEffectiveness": 0.35,
    "equityNeed": 0.3,
    "controllability": 0.2,
    "rotationFairness": 0.15
  }
}
```

```
{
  "eventId": "evt_001",
  "targetFlexEnergyKwh": 24.0,
  "selected": [
    {
      "resourceId": "HW-101",
      "type": "STORAGE_HOT_WATER",
      "score": 0.91,
      "maxShiftEnergyKwh": 5.0,
      "scheduledEnergyKwh": 5.0,
      "decision": "SELECTED",
      "scoreBreakdown": {
        "networkEffectiveness": 0.95,
        "equityNeed": 0.9,
        "controllability": 1.0,
        "rotationFairness": 0.8
      }
    },
    {
      "resourceId": "HW-102",
      "type": "STORAGE_HOT_WATER",
      "score": 0.86,
      "maxShiftEnergyKwh": 4.0,
      "scheduledEnergyKwh": 4.0,
      "decision": "SELECTED"
    },
    {
      "resourceId": "BAT-01",
      "type": "COMMUNITY_BATTERY",
      "score": 0.78,
      "maxShiftEnergyKwh": 10.0,
      "scheduledEnergyKwh": 10.0,
      "decision": "SELECTED"
    },
    {
      "resourceId": "APT-01",
      "type": "APARTMENT_COMMON_LOAD",
      "score": 0.76,
      "maxShiftEnergyKwh": 6.0,
      "scheduledEnergyKwh": 5.0,
      "decision": "PARTIALLY_SELECTED"
    }
  ],
  "rejected": [
    {
      "resourceId": "EV-04",
      "decision": "REJECTED",
      "reason": "UNAVAILABLE"
    },
    {
      "resourceId": "HW-205",
      "decision": "REJECTED",
      "reason": "WRONG_CELL"
    }
  ]
}
```

```
"scheduledFlexEnergyKwh": 24.0,  
"remainingTargetKwh": 0.0,  
"status": "OPTIMISED"  
}
```

Optimiser calculation

```
score = 0.35*networkEffectiveness  
       + 0.30*equityNeed  
       + 0.20*controllability  
       + 0.15*rotationFairness  
  
scheduledEnergyKwh = min(resource.maxShiftEnergyKwh, remainingTargetKwh)
```

Hard gates first

A resource with no consent, wrong Sunshine Cell, unavailable state, incompatible control or unsafe comfort condition is rejected before scoring. A high equity score can never override a hard technical/safety rule.

11. Simulated Dispatch API

POST /api/v1/events/:eventId/simulate [SIMULATED]

Purpose: Represent partner dispatch and deterministic device response. External control is simulated; response calculation is real code.

Used by: Council Run Simulated Event button

```
{
  "simulationSeed": "DEMO_V1",
  "useDeterministicResponseFactors": true
}
```

```
{
  "eventId": "evt_001",
  "status": "DISPATCHED",
  "expectedFlexEnergyKwh": 24.0,
  "resourceResponses": [
    {
      "resourceId": "HW-101",
      "scheduledEnergyKwh": 5.0,
      "responseFactor": 0.96,
      "actualShiftEnergyKwh": 4.8,
      "partnerAck": "SIMULATED_ACCEPTED"
    },
    {
      "resourceId": "HW-102",
      "scheduledEnergyKwh": 4.0,
      "responseFactor": 0.975,
      "actualShiftEnergyKwh": 3.9,
      "partnerAck": "SIMULATED_ACCEPTED"
    },
    {
      "resourceId": "BAT-01",
      "scheduledEnergyKwh": 10.0,
      "responseFactor": 0.97,
      "actualShiftEnergyKwh": 9.7,
      "partnerAck": "SIMULATED_ACCEPTED"
    },
    {
      "resourceId": "APT-01",
      "scheduledEnergyKwh": 5.0,
      "responseFactor": 0.84,
      "actualShiftEnergyKwh": 4.2,
      "partnerAck": "SIMULATED_ACCEPTED"
    }
  ],
  "simulatedActualFlexEnergyKwh": 22.6,
  "provenance": "SIMULATED"
}
```

Why deterministic

The same inputs always produce the same 22.6 kWh result. The demo is reliable, while still showing that scheduled energy (24.0 kWh) is not automatically equal to delivered response.

12. Measurement & Verification (M&V) API

POST /api/v1/events/:eventId/verify [REAL]

Purpose: Estimate the no-intervention baseline, compare it with observed event energy, calculate verified flexibility and gate settlement by confidence.

Used by: Verify & Settle screen

```
{
  "baselineMethod": "AVERAGE_COMPARABLE_NON_EVENT_DAYS",
  "minimumComparableDays": 3,
  "minimumSettlementConfidence": 0.7
}
```

```
{
  "eventId": "evt_001",
  "baseline": {
    "method": "AVERAGE_COMPARABLE_NON_EVENT_DAYS",
    "referenceDays": [
      {
        "date": "2026-08-15",
        "eventRan": false,
        "energyKwh": 49.0
      },
      {
        "date": "2026-08-16",
        "eventRan": false,
        "energyKwh": 51.0
      },
      {
        "date": "2026-08-19",
        "eventRan": false,
        "energyKwh": 50.0
      }
    ],
    "baselineEnergyKwh": 50.0,
    "provenance": "ESTIMATED"
  },
  "observation": {
    "observedEnergyKwh": 72.6,
    "provenance": "SIMULATED"
  },
  "verifiedFlexEnergyKwh": 22.6,
  "mvConfidence": 0.82,
  "minimumSettlementConfidence": 0.7,
  "settlementAllowed": true,
  "status": "VERIFIED"
}
```

```
baselineEnergyKwh = average(comparableDays WHERE eventRan == false)
verifiedFlexEnergyKwh = max(0, observedEnergyKwh - baselineEnergyKwh)
settlementAllowed = mvConfidence >= minimumSettlementConfidence
```

Baseline contamination rule

Previous Sunshine Wallet event periods must be excluded from the simple MVP baseline. Otherwise the baseline drifts upward toward event consumption and underestimates the intervention.

13. Contributor Attribution API

GET /api/v1/events/:eventId/contributor-attribution [REAL]

Purpose: Calculate each opted-in solar contributor's share of qualifying export and cap total attribution at the verified accommodated amount.

Used by: Settlement engine + contributor wallet

```
{
  "eventId": "evt_001",
  "qualifyingExports": [
    {
      "participantId": "p_maria",
      "measuredExportKwh": 6.5,
      "qualifyingExportKwh": 6.5
    },
    {
      "participantId": "p_alex",
      "measuredExportKwh": 4.0,
      "qualifyingExportKwh": 4.0
    },
    {
      "participantId": "p_sam",
      "measuredExportKwh": 2.5,
      "qualifyingExportKwh": 2.5
    }
  ],
  "totalQualifyingExportKwh": 13.0,
  "verifiedAccommodationKwh": 10.0,
  "attributableAccommodationKwh": 10.0,
  "contributors": [
    {
      "participantId": "p_maria",
      "share": 0.5,
      "attributedAccommodationKwh": 5.0
    },
    {
      "participantId": "p_alex",
      "share": 0.307692,
      "attributedAccommodationKwh": 3.07692
    },
    {
      "participantId": "p_sam",
      "share": 0.192308,
      "attributedAccommodationKwh": 1.92308
    }
  ]
}
```

```
share_i = qualifyingExportKwh_i / totalQualifyingExportKwh
attributableAccommodationKwh = min(totalQualifyingExportKwh, verifiedAccommodationKwh)
attributed_i = share_i * attributableAccommodationKwh
```

No electron tracing

This is accounting attribution. The API must never claim that Maria's exact electrons went to Dinh. It records Maria's export during the same qualifying event and relevant Sunshine Cell, then allocates the verified contributor outcome proportionally.

14. Settlement API

POST /api/v1/events/:eventId/settle [REAL]

Purpose: Convert verified event value into Equity Pool, Contributor Pool and reserve; enforce Equity Floor; allocate household and contributor shares.

Used by: Council Verify & Settle screen

```
{
  "policy": {
    "version": "policy_v1",
    "settlementRateCentsPerVerifiedKwh": 80,
    "equityFloor": 0.6,
    "equityPoolShare": 0.65,
    "contributorPoolShare": 0.3,
    "communityReserveShare": 0.05
  },
  "minimumMvConfidence": 0.7
}
```

```
{
  "eventId": "evt_001",
  "verifiedFlexEnergyKwh": 22.6,
  "verifiedProgramValueCents": 1808,
  "policyValidation": {
    "equityFloor": 0.6,
    "equityPoolShare": 0.65,
    "valid": true
  },
  "pools": {
    "equityPoolCents": 1175,
    "contributorPoolCents": 542,
    "communityReserveCents": 91
  },
  "equityAllocations": [
    {
      "participantId": "p_dinh",
      "weight": 0.35,
      "amountCents": 411
    },
    {
      "participantId": "p_sarah",
      "weight": 0.35,
      "amountCents": 411
    },
    {
      "participantId": "p_james",
      "weight": 0.3,
      "amountCents": 353
    }
  ],
  "contributorAllocations": [
    {
      "participantId": "p_maria",
      "share": 0.5,
      "amountCents": 271
    },
    {
      "participantId": "p_alex",
      "share": 0.307692,
      "amountCents": 167
    },
    {
      "participantId": "p_sam",
      "share": 0.192308,
      "amountCents": 104
    }
  ],
  "status": "SETTLED",
  "provenance": {
    "settlementRate": "SIMULATED",
    "allocationMath": "CALCULATED"
  }
}
```

Rounding rule

Calculate pools in integer cents. Allocate remainder cents deterministically (for example, largest-remainder method or stable participant ID order) so all allocations sum exactly to the program value.

15. Credit Ledger API

POST /api/v1/events/:eventId/apply-credits [REAL]

Purpose: Write calculated settlement allocations to participant wallets exactly once.

Used by: Council final action + resident/contributor wallets

Header: Idempotency-Key: evt_001-credit-apply-v1

```
{
  "eventId": "evt_001",
  "apply": true
}
```

```
{
  "eventId": "evt_001",
  "status": "CREDITS_APPLIED",
  "credits": [
    {
      "creditId": "cr_001",
      "participantId": "p_dinh",
      "type": "EQUITY_DIVIDEND",
      "amountCents": 411,
      "status": "APPLIED"
    },
    {
      "creditId": "cr_002",
      "participantId": "p_maria",
      "type": "CONTRIBUTOR_REWARD",
      "amountCents": 271,
      "status": "APPLIED"
    }
  ],
  "idempotency": {
    "key": "evt_001-credit-apply-v1",
    "replayed": false
  }
}
```

Why idempotency matters

If the presenter double-clicks or the browser retries the request, Dinh and Maria must not receive duplicate credits. This is a tiny implementation detail with high credibility.

POST /api/v1/events/:eventId/reset [SIMULATED]

Purpose: Return the demo event and wallet fixtures to their known starting state. Never expose this in a production API.

Used by: Hackathon operator/demo control

```
{
  "resetProfile": "DEMO_V1"
}
```

16. Canonical Event Read Model

GET /api/v1/events/:eventId [REAL]

Purpose: Return one complete read model so screens can refresh safely without reconstructing state from many calls.

Used by: Council UI

```
{
  "event": {
    "id": "evt_001",
    "sunshineCellId": "cell_dapto_01",
    "startTime": "2026-08-23T12:00:00+10:00",
    "endTime": "2026-08-23T14:00:00+10:00",
    "targetFlexEnergyKwh": 24.0,

```



```

    "availableFlexEnergyKwh": 31.0,
    "forecastConfidence": 0.82,
    "status": "CREDITS_APPLIED"
  },
  "optimisation": {
    "scheduledFlexEnergyKwh": 24.0,
    "selectedCount": 4,
    "rejectedCount": 2
  },
  "simulation": {
    "actualFlexEnergyKwh": 22.6
  },
  "verification": {
    "baselineEnergyKwh": 50.0,
    "observedEnergyKwh": 72.6,
    "verifiedFlexEnergyKwh": 22.6,
    "mvConfidence": 0.82,
    "settlementAllowed": true
  },
  "settlement": {
    "verifiedProgramValueCents": 1808,
    "equityPoolCents": 1175,
    "contributorPoolCents": 542,
    "reserveCents": 91
  },
  "provenance": {
    "network": "SIMULATED",
    "meter": "SIMULATED",
    "settlementRate": "SIMULATED"
  }
}

```

17. Error Contract

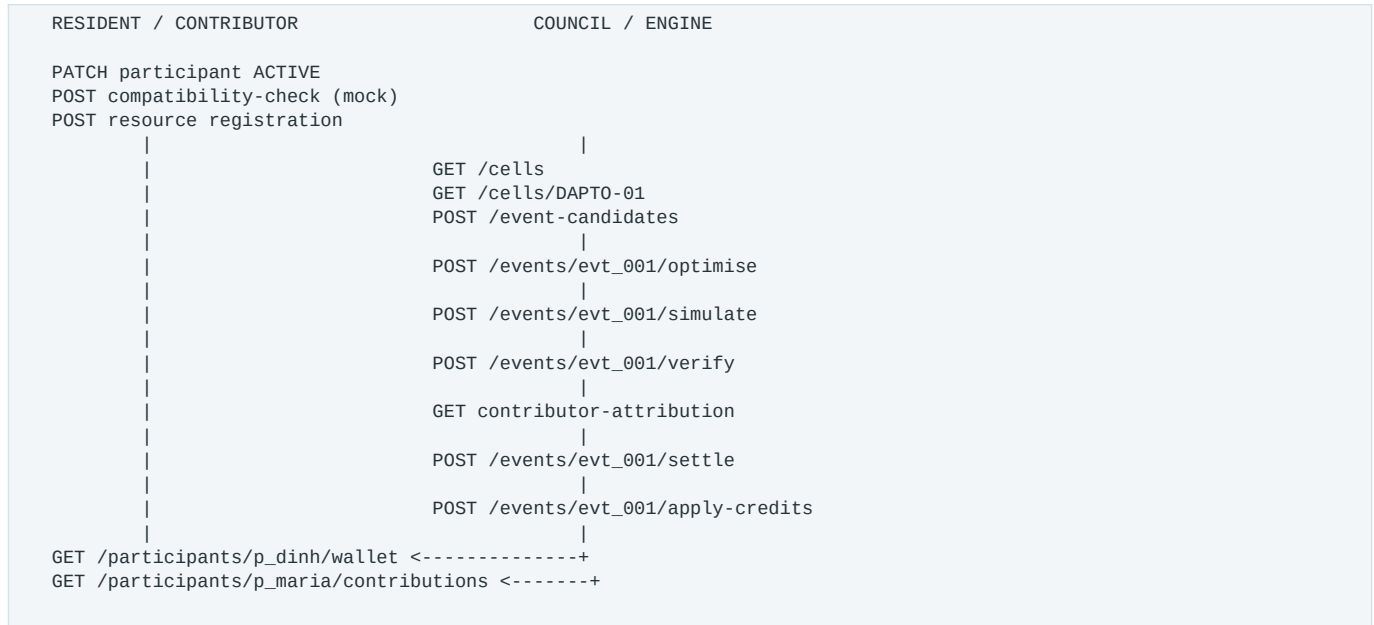
```
{
  "error": {
    "code": "INVALID_EVENT_STATE",
    "message": "Event must be OPTIMISED before simulation.",
    "details": {
      "eventId": "evt_001",
      "currentStatus": "PROPOSED",
      "requiredStatus": "OPTIMISED"
    },
    "requestId": "req_abc123"
  }
}
```

HTTP	Code	When
400	INVALID_REQUEST	Missing/invalid fields, units, ranges or malformed timestamps.
404	NOT_FOUND	Participant, resource, cell or event does not exist.
409	INVALID_EVENT_STATE	Action called in wrong state.
409	DUPLICATE_CREDIT_APPLICATION	Credit apply attempted without matching idempotency semantics.
409	EQUITY_FLOOR_VIOLATION	Proposed settlement allocates less than required equity share.
422	INSUFFICIENT_FLEXIBILITY	Eligible resources cannot meet minimum event requirement.
422	LOW_FORECAST_CONFIDENCE	Event should not dispatch.
422	LOW_MV_CONFIDENCE	Result may be displayed, but settlement is blocked.
422	NO_BASELINE	Insufficient comparable non-event reference days.
503	PARTNER MOCK_UNAVAILABLE	Demo partner adapter intentionally failed; frontend can show fallback.

18. Validation Rules

Field/rule	Validation
targetFlexEnergyKwh	≥ 0 ; energy unit only.
maxPowerKw	> 0 ; never used as event energy target.
maxShiftEnergyKwh	≥ 0 and limited by device/event constraints.
confidence	$0 \leq \text{value} \leq 1$.
responseFactor	$0 \leq \text{value} \leq 1$ for MVP deterministic simulation.
pool shares	Each 0..1; sum exactly 1.0 before rounding.
equityPoolShare	Must be $\geq \text{equityFloor}$.
event times	$\text{endTime} > \text{startTime}$; same Sunshine Cell timezone.
settlement	Blocked unless status VERIFIED and settlementAllowed = true.
credits	Integer cents; sum of allocations equals pool total exactly.

19. End-to-End API Sequence



Demo navigation

The "Simulate Flex Event" button should call /simulate, then /verify (or navigate while those requests resolve), and open the Verify & Settle screen. "Apply Credits & View Resident Wallet" calls /apply-credits and then routes to Dinh's wallet.

20. Frontend-to-Endpoint Mapping

Screen	Calls
Dinh wallet	GET /participants/p_dinh/wallet; PATCH participation
Maria contributor wallet	GET /participants/p_maria/wallet; GET /contributions; PATCH participation
Device onboarding	POST /resources/compatibility-check; POST /resources
Council opportunity dashboard	GET /cells; GET /cells/:id; POST /event-candidates
Optimise DAPTO-01	POST /events/:id/optimise; GET /events/:id
Verify & Settle	POST /events/:id/simulate; POST /events/:id/verify; GET contributor-attribution; POST /settle
Final wallet loop	POST /apply-credits; GET participant wallet

21. MVP Persistence Model

For the 25-hour build, the same API contract can be backed by seeded TypeScript/JSON. The contract should not depend on whether Supabase is added later.

Fixture/table	Minimum fields
participants	id, householdId, displayName, role, equityEligible, participationStatus, sunshineCellId
resources	id, householdId, cellId, type, maxPowerKw, maxShiftEnergyKwh, availability, compatibility, consent, responseFactor, score inputs
contributors	participantId, cellId, participationStatus, export profile
cells	id, displayName, suburb, topologyRef, equity/solar/network scores, provenance
energyIntervals	cellId, timestamp, forecastExportEnergyKwh, comfortableExportEnergyKwh, baseline/observed demand fields, confidence
events	id, cellId, start/end, target energy, confidence, status
dispatches	eventId, resourceId, score, scheduled, actual, decision, reason
verifications	eventId, baseline, observed, verified, confidence, settlementAllowed
settlements	eventId, valueCents, policyVersion, pool totals
credits	id, eventId, participantId, type, amountCents, status

22. Suggested Next.js Route Layout

```

app/api/v1/
  participants/[participantId]/route.ts
  participants/[participantId]/participation/route.ts
  participants/[participantId]/wallet/route.ts
  participants/[participantId]/contributions/route.ts

  resources/route.ts
  resources/compatibility-check/route.ts

  cells/route.ts
  cells/[cellId]/route.ts
  cells/[cellId]/event-candidates/route.ts

  events/[eventId]/route.ts
  events/[eventId]/optimise/route.ts
  events/[eventId]/simulate/route.ts
  events/[eventId]/verify/route.ts
  events/[eventId]/contributor-attribution/route.ts
  events/[eventId]/settle/route.ts
  events/[eventId]/apply-credits/route.ts
  events/[eventId]/reset/route.ts

lib/engine/
  selectEventWindow.ts
  checkResourceEligibility.ts
  optimiseResources.ts
  simulateEvent.ts
  calculateBaseline.ts
  verifyEvent.ts
  attributeContributors.ts
  settleEvent.ts

lib/contracts/
  types.ts
  errors.ts
  schemas.ts

```

23. Real vs Mock Boundary

Capability	MVP implementation	Production replacement
Device compatibility	Mock API response	Authorised retailer/metering-provider/aggregator integration
Device dispatch	Simulated acknowledgement	Partner control API
Smart-meter demand	Seeded interval readings	Authorised interval-meter data
Solar export	Seeded contributor readings	Meter/inverter/retailer data
Sunshine Cell topology	Synthetic DAPTO-01	DNSP topology / transformer / phase mapping
Network constraint	Simulated export opportunity and comfortable level	DNSP network telemetry/planning data
Baseline	Average simulated comparable non-event days	Validated production M&V methodology
Settlement rate	Simulated cents/kWh program value	Agreed retailer/network/aggregator commercial product
Optimiser	Real TypeScript function	Can evolve, but transparent constraints should remain
Equity Floor	Real policy validation	Governed program policy
Credit ledger	Real demo ledger state	Retailer bill-credit / formal settlement integration

24. P0 Contract - What Must Work in the Demo

- GET DAPTO-01 state returns 24 kWh target, 31 kWh available flexibility and 82% forecast confidence.
- POST optimise selects exactly enough energy, partially selecting APT-01 and rejecting EV-04/HW-205 with reasons.
- POST simulate deterministically converts 24.0 kWh scheduled into 22.6 kWh actual response.
- POST verify calculates baseline from non-event days and returns 22.6 kWh verified flexibility with a confidence gate.
- Contributor attribution records Maria's export share without claiming electron tracing.
- POST settle enforces the Equity Floor and calculates integer-cent pools/allocations.
- POST apply-credits is idempotent and updates Dinh/Maria wallet read models.
- Every external-infrastructure field is visibly marked MOCKED/SIMULATED/ESTIMATED.

Minimum successful API story

The frontend can start with static fixture responses, but by the integration checkpoint these endpoints must call real engine functions. Changing an input (resource availability, Equity Floor, contributor export, confidence) must change the calculated output.

25. Contract Test Cases

Test	Expected result
Happy path	24 kWh target -> 24 scheduled -> 22.6 actual -> verified -> valid settlement -> credits applied.
EV becomes available	Eligibility/optimiser may rank/select EV depending on score and remaining target.
HW-205 moved into DAPTO-01	Wrong-cell rejection disappears and resource becomes scoreable.
Forecast confidence = 0.60	No event dispatch; LOW_FORECAST_CONFIDENCE.
M&V confidence = 0.63	Verification shown; settlementAllowed=false; LOW_MV_CONFIDENCE on settle.
Equity pool share = 0.50, floor = 0.60	EQUITY_FLOOR_VIOLATION; no settlement.
No clean baseline days	NO_BASELINE or lower-confidence provisional result; no credits.
Maria export drops from 6.5 to 2.0 kWh	Her contributor share and reward fall automatically.
Double apply-credits with same idempotency key	Same response returned; no duplicate credit.
Reset demo	All event/wallet state returns to fixture baseline.

26. Implementation Priority for Three Developers

Owner	Contract responsibility
Developer 1 - engine/API	Types, fixtures, /event-candidates, /optimise, /simulate, /verify, attribution, /settle, /apply-credits, contract tests.
Developer 2 - Council UI	Build against the exact event/read-model JSON; no waiting for engine. Replace fixtures with live routes during integration.
Developer 3 - Resident/contributor UI	Wallet, participation, compatibility mock and contribution views using exact participant/wallet contracts.

Freeze the contract early

Do not casually rename fields during the hackathon. If a contract change is necessary, Developer 1 updates types + sample JSON + endpoint response in one commit, and the team announces the breaking change before merge.

27. Final Contract Checklist

- All energy fields say Kwh; all instantaneous power fields say Kw.
- All confidence fields use decimals 0..1 in JSON.
- All money writes use integer cents.
- Every response that contains simulated/estimated data includes provenance.
- Event state transitions reject invalid orderings.
- Optimiser rejection reasons are machine-readable enums plus human-readable UI copy.
- M&V excludes previous event periods from the simple baseline.
- M&V confidence can block settlement without hiding the provisional result.
- Contributor attribution is proportional accounting, not electron tracing.
- Equity Floor is enforced before settlement.
- Apply-credits is idempotent.
- Demo reset is available and deterministic.
- Frontend screens can render from the canonical GET /events/:id and wallet endpoints after refresh.

One-sentence API principle

Mock the infrastructure, not the logic: the API must make every important decision, calculation, rejection, confidence gate and credit allocation explicit and reproducible.